

Experiment-Fork() : Command Execution using fork(), exec(), and wait()

Program 1: One Process (Before fork())

```
#include <stdio.h>

#include <unistd.h>

int main()

{ printf("PID = %d\n", getpid());

return 0;

}
```

Only one process exists.

Introduce getpid()

```
aasritha@DESKTOP-4B676UM:~$ cd OSLab
aasritha@DESKTOP-4B676UM:~/OSLab$ mkdir fork
mkdir: cannot create directory 'fork': File exists
aasritha@DESKTOP-4B676UM:~/OSLab$ cd fork
aasritha@DESKTOP-4B676UM:~/OSLab/fork$ nano program1.c
aasritha@DESKTOP-4B676UM:~/OSLab/fork$ gcc program1.c -o program1
aasritha@DESKTOP-4B676UM:~/OSLab/fork$ ./program1
PID = 1671
aasritha@DESKTOP-4B676UM:~/OSLab/fork$ |
```

Program 2: fork() Creates a Child

```
#include <stdio.h>

#include <unistd.h>

int main() {

fork();

printf("Hello\n");

return 0;

}
```

```
aasritha@DESKTOP-4B676UM:~/OSLab/fork$ nano program2.c
aasritha@DESKTOP-4B676UM:~/OSLab/fork$ gcc program2.c -o program2
aasritha@DESKTOP-4B676UM:~/OSLab/fork$ ./program2
Hello
Hello
aasritha@DESKTOP-4B676UM:~/OSLab/fork$ |
```

Program 3: Parent vs Child

```
#include <stdio.h>

#include <unistd.h>

int main() {
    pid_t pid;
    pid = fork();
    if(pid == 0)
        printf("I am Child\n");
    else
        printf("I am Parent\n");
    return 0;
}
```

```
aasritha@DESKTOP-4B676UM:~/OSLab/fork$ nano program3.c
aasritha@DESKTOP-4B676UM:~/OSLab/fork$ gcc program3.c -o program3
aasritha@DESKTOP-4B676UM:~/OSLab/fork$ ./program3
I am Parent
I am Child
aasritha@DESKTOP-4B676UM:~/OSLab/fork$ |
```

Program 4: Display Both PIDs

```
#include <stdio.h>

#include <unistd.h>

int main()
{ pid_t pid;
  pid = fork();
```

```

if(pid == 0) {
    printf("Child PID = %d\n", getpid());
    printf("Parent PID = %d\n", getppid());
} else {
    printf("Parent PID = %d\n", getpid());
    printf("Child PID = %d\n", pid);
}
return 0;
}

```

```

aasritha@DESKTOP-4B676UM:~/OSLab/fork$ nano program4.c
aasritha@DESKTOP-4B676UM:~/OSLab/fork$ gcc program4.c -o program4
aasritha@DESKTOP-4B676UM:~/OSLab/fork$ ./program4
Parent PID = 1695
Child PID = 1696
Child PID = 1696
Parent PID = 1695
aasritha@DESKTOP-4B676UM:~/OSLab/fork$ |

```

Program 5: Parent Waits for Child

```

#include <stdio.h>

#include <unistd.h>

#include <sys/wait.h>

int main() {
    pid_t pid = fork();
    if(pid == 0) {
        printf("Child Executing\n");
    } else {
        wait(NULL);
    }
}

```

```

        printf("Parent Executing\n");
    }
    return 0;
}

```

```

aasritha@DESKTOP-4B676UM:~$ cd OSLab
aasritha@DESKTOP-4B676UM:~/OSLab$ cd fork
aasritha@DESKTOP-4B676UM:~/OSLab/fork$ nano program5.c
aasritha@DESKTOP-4B676UM:~/OSLab/fork$ gcc program5.c -o program5
aasritha@DESKTOP-4B676UM:~/OSLab/fork$ ./program5
Child Executing
Parent Executing
aasritha@DESKTOP-4B676UM:~/OSLab/fork$ |

```

Program 6: Multiple fork() Calls

```

#include <stdio.h>

#include <unistd.h>

int main() {
    fork();
    fork();
    printf("PID = %d\n", getpid());
    return 0;
}

```

```

aasritha@DESKTOP-4B676UM:~/OSLab/fork$ nano program6.c
aasritha@DESKTOP-4B676UM:~/OSLab/fork$ gcc program6.c -o program6
aasritha@DESKTOP-4B676UM:~/OSLab/fork$ ./program6
PID = 36
PID = 37
PID = 38
PID = 39
aasritha@DESKTOP-4B676UM:~/OSLab/fork$ |

```

Program 7: Child Executes Another Program (exec())

```

#include <stdio.h>

#include <unistd.h>

int main() {

```

```

pid_t pid = fork();

if(pid == 0) {
    printf("Child executing ls...\n");
    execl("/bin/ls", "ls", "-l", NULL);
} else {
    printf("Parent Waiting...\n");
}

return 0;
}

```

```

aasritha@DESKTOP-4B676UM:~/OSLab/fork$ nano program7.c
aasritha@DESKTOP-4B676UM:~/OSLab/fork$ gcc program7.c -o program7
aasritha@DESKTOP-4B676UM:~/OSLab/fork$ ./program7
Parent Waiting...
Child executing ls...
aasritha@DESKTOP-4B676UM:~/OSLab/fork$ total 140
-rwxr-xr-x 1 aasritha aasritha 16008 Jul 31 08:35 program1
-rw-r--r-- 1 aasritha aasritha  102 Jul 31 08:35 program1.c
-rwxr-xr-x 1 aasritha aasritha 16008 Jul 31 08:44 program2
-rw-r--r-- 1 aasritha aasritha   98 Jul 31 08:43 program2.c
-rwxr-xr-x 1 aasritha aasritha 16008 Jul 31 09:13 program3
-rw-r--r-- 1 aasritha aasritha  167 Jul 31 09:12 program3.c
-rwxr-xr-x 1 aasritha aasritha 16096 Jul 31 09:14 program4
-rw-r--r-- 1 aasritha aasritha  282 Jul 31 09:14 program4.c
-rwxr-xr-x 1 aasritha aasritha 16048 Jul 31 09:57 program5
-rw-r--r-- 1 aasritha aasritha  219 Jul 31 09:56 program5.c
-rwxr-xr-x 1 aasritha aasritha 16048 Jul 31 09:58 program6
-rw-r--r-- 1 aasritha aasritha  121 Jul 31 09:58 program6.c
-rwxr-xr-x 1 aasritha aasritha 16048 Jul 31 10:00 program7
-rw-r--r-- 1 aasritha aasritha  224 Jul 31 10:00 program7.c

```

Program 8: fork() Inside a Loop (Fun)

```

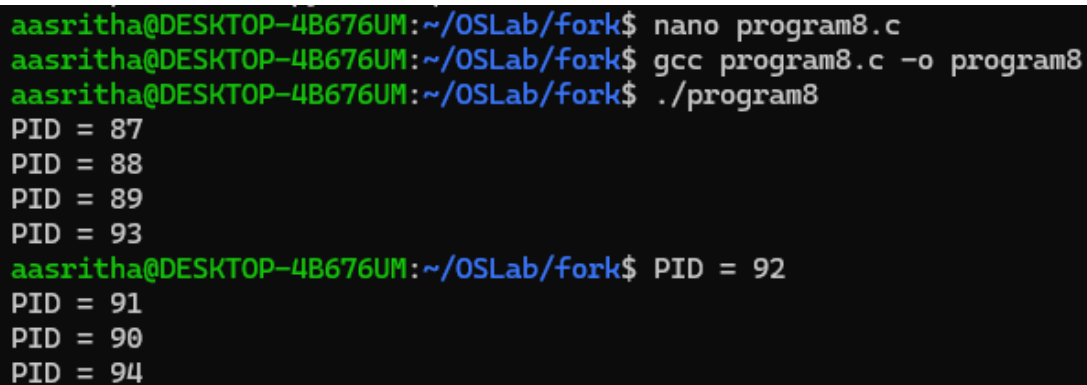
#include <stdio.h>

#include <unistd.h>

int main() {

```

```
int i;
for(i=0;i<3;i++)
fork();
printf("PID = %d\n",getpid());
return 0;
}
```

A terminal window with a black background and green text. The prompt is 'aasritha@DESKTOP-4B676UM:~/OSLab/fork\$'. The user enters 'nano program8.c', then 'gcc program8.c -o program8', and finally './program8'. The output shows four lines of 'PID = 87', 'PID = 88', 'PID = 89', and 'PID = 93'. Then the user enters 'PID = 92', and the output shows three lines of 'PID = 91', 'PID = 90', and 'PID = 94'.

```
aasritha@DESKTOP-4B676UM:~/OSLab/fork$ nano program8.c
aasritha@DESKTOP-4B676UM:~/OSLab/fork$ gcc program8.c -o program8
aasritha@DESKTOP-4B676UM:~/OSLab/fork$ ./program8
PID = 87
PID = 88
PID = 89
PID = 93
aasritha@DESKTOP-4B676UM:~/OSLab/fork$ PID = 92
PID = 91
PID = 90
PID = 94
```

Program 9

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/wait.h>
int main() {
char cmd[50];
printf("Enter Linux Command: ");
scanf("%s", cmd);
pid_t pid = fork();
if(pid < 0) {
    printf("Fork Failed\n");
    return 1;
} else if(pid == 0) {
```

```

printf("\nChild Process\n");
printf("Child PID : %d\n", getpid());
printf("Parent PID: %d\n", getppid());
execlp(cmd, cmd, NULL);
perror("Command Execution Failed");
exit(1);
} else {
    printf("\nParent Process\n");
    printf("Parent PID : %d\n", getpid());
    printf("Child PID : %d\n", pid);
    wait(NULL);
    printf("\nChild Process Completed\n");
}
return 0;
}

```

```

aasritha@DESKTOP-4B676UM:~/OSLab/fork$ nano program9.c
aasritha@DESKTOP-4B676UM:~/OSLab/fork$ gcc program9.c -o program9
aasritha@DESKTOP-4B676UM:~/OSLab/fork$ ./program9
Enter Linux Command: ls

Parent Process
Parent PID : 110
Child PID : 111

Child Process
Child PID : 111
Parent PID: 110
program1  program2  program3  program4  program5  program6  program7  program8  program9
program1.c program2.c program3.c program4.c program5.c program6.c program7.c program8.c program9.c

Child Process Completed
aasritha@DESKTOP-4B676UM:~/OSLab/fork$ |

```